EECS 484, Winter 2026 - Database Management Systems

Mapping ER to Relational

Chapter 3.5

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

Last Time

Relational Model

- A tabular representation of data
- Simple and intuitive
- Currently the most widely used database model
- Integrity constraints specified by DBA. DBMS checks for violations.
  - Two important ICs: **primary** and **foreign keys**
  - We **always** have domain constraints

e.g. `INTEGER` fields must always contain integer values

Students

sid	name	login	age
13	Lisa	lsimp	40
41	Bart	bart	20

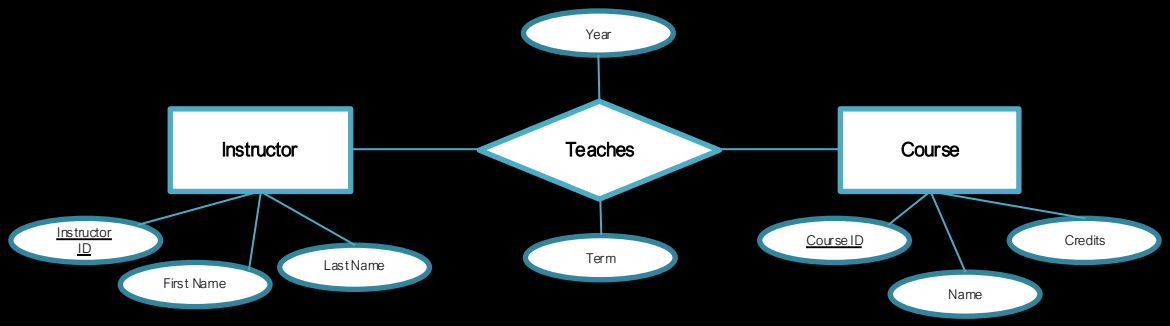
Courses

cid	cname	Cr.
E-484	EECS484	4
E-584	EECS584	3

Enrolled

sid	cid	Grade
41	E-484	A-
13	E-584	A+

From ER To Relational Model



Instructor ID	First Name	Last Name
394953	John	Smith
454544	Sara	King
439849	Alex	Dee
....	....	....
....	....	....

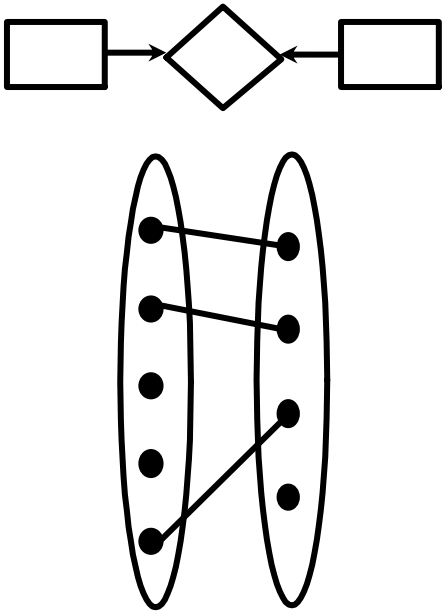
Instructor ID	Course ID	Year	Term
454544	E302	2009	F
394953	C210	2010	W
439849	M184	2010	F
....	....	....	....
....	....	....	....

Course ID	Name	Credits
M184	Calculus	3
C210	Physics	4
E302	Algorithms	4
....	....	....
....	....	....

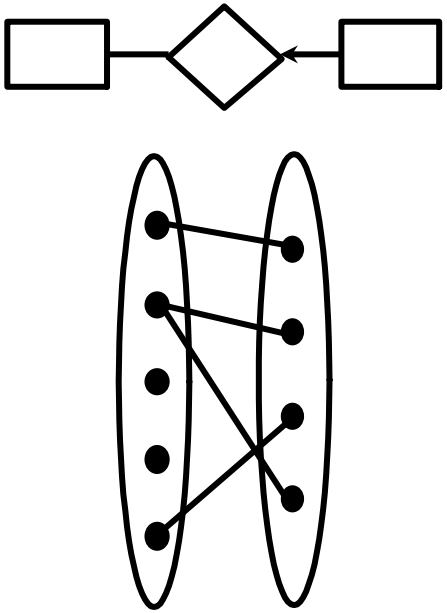
Recall ER Constructs

- Basic Constructs
  - Entity Sets
  - Relationship Sets
  - Attributes (of entities and relationships)
- Additional Constructs
  - ISA Hierarchies
  - Weak Entities
  - Aggregation
- Integrity Constraints
  - Key constraints
  - Participation constraints
  - Overlap / Covering constraints for ISA hierarchies

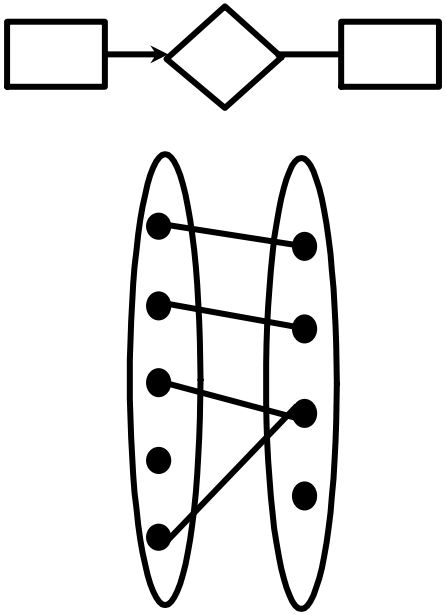
Review: ER relationship types



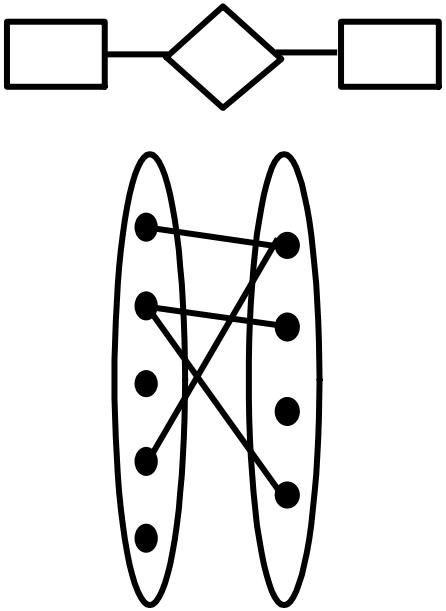
1-to-1



1-to Many
1-to-N



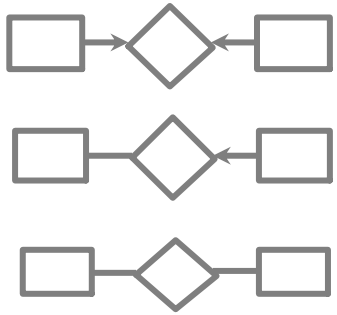
Many-to-1
N-to-1



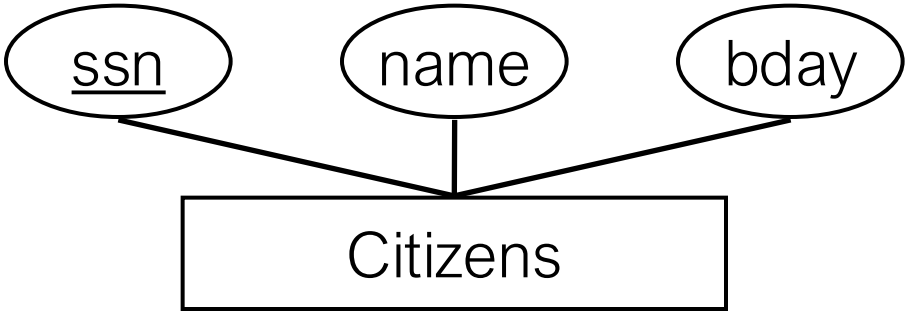
Many-to-Many
M-to-N

ER to Tables: Basics

- **Strong entities:**
  - key = primary key
- **Binary relationships:** keys come from the participating entities
  - **1-to-1:** either key (other = candidate key)
  - **1-to-N:** the key of the 'many' (N) part
  - **M-to-N:** both keys



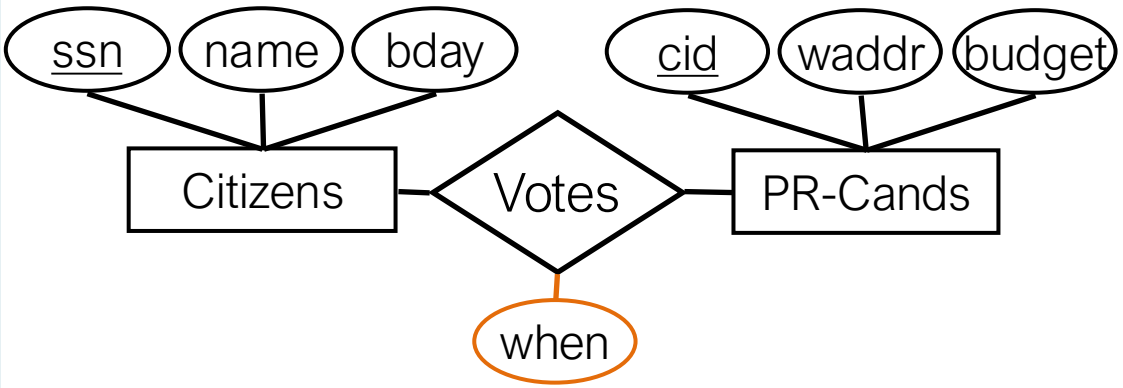
Entity Sets to Tables




```
CREATE TABLE Citizens
    (ssn    CHAR(11),
     name   CHAR(20),
     bday   DATE,
     PRIMARY KEY  (ssn))
```

Can ssn have a null value?

Relationship Sets to Tables



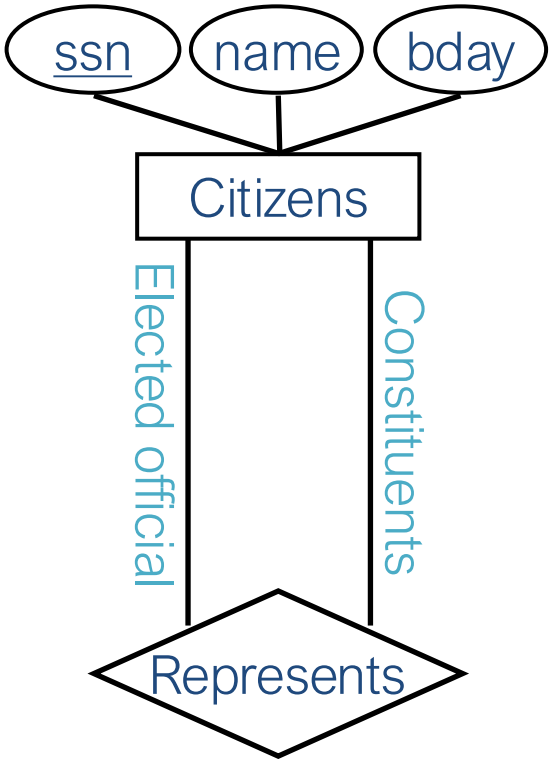

```
CREATE TABLE Votes(
        ssn       CHAR(11),
        cid       INTEGER,
        when      DATE,
PRIMARY KEY (ssn, cid),
FOREIGN KEY (ssn) REFERENCES Citizens (ssn),
FOREIGN KEY (cid) REFERENCES PR-Cands (cid))
```

- **Foreign key: keys from** participating entity sets
- Descriptive attributes

Generalizes to n-ary relationships

(we will see example later)

Relationship Sets to Tables

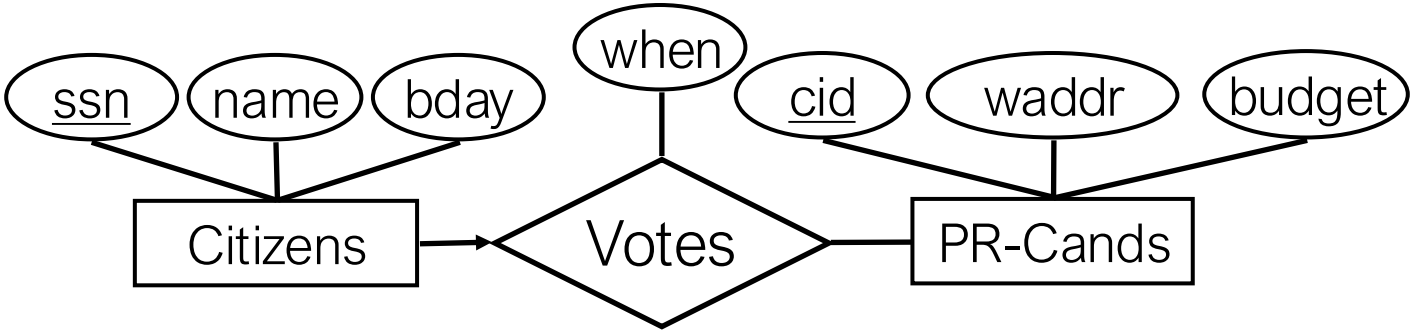



```
CREATE TABLE Represents(
        elected_ssn  CHAR(11),
        cons_ssn     CHAR(11),
PRIMARY KEY (elected_ssn, cons_ssn),
FOREIGN KEY (elected_ssn) REFERENCES Citizens(ssn),
FOREIGN KEY (cons_ssn) REFERENCES Citizens(ssn))
```

Note that you need to specify the column that you are referring to in the `Citizens` table, as `Citizens` does not have "`elected_ssn`" nor "`cons_ssn`".

BRACE YOURSELVES
Many new slides are coming
Troll.me

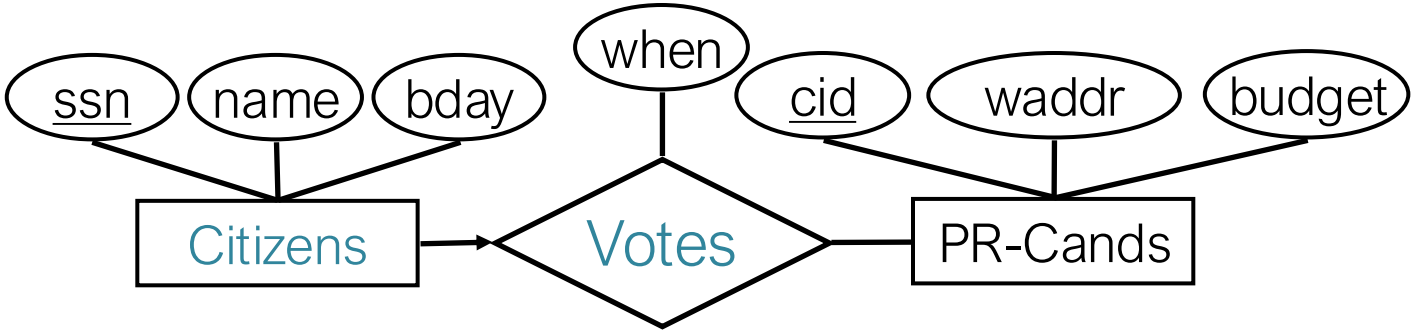
Key Constraints



- Approach 1: **Three** Tables (Citizens, Votes, PR-Cands)
  - But Votes P.K. must be ssn and not (ssn,cid)

```
CREATE TABLE  Votes
(   ssn     CHAR(11),
    cid     INTEGER,
    when    DATE,
    PRIMARY KEY (ssn),
    FOREIGN KEY (ssn) REFERENCES Citizens (ssn),
    FOREIGN KEY (cid) REFERENCES PR-Cands (cid))
```

Key Constraints



Each citizen can only vote once, so OK to fold 'Votes' relationship into 'Citizens' entity

- Approach 2: **Two** Tables (Citizen_Votes, PR-Cands)

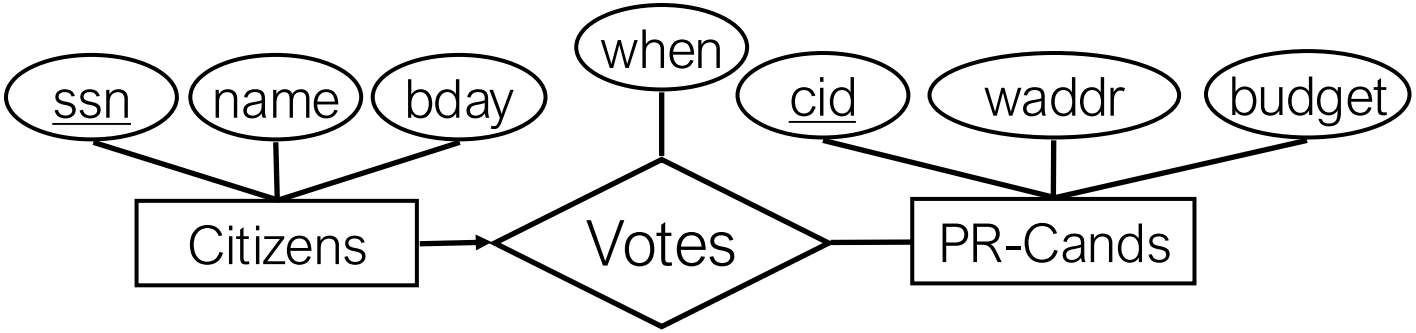
```
CREATE TABLE  Citizen_Votes (
    ssn      CHAR(11),
    name     CHAR(20),
    bday     DATE,
    when     DATE,
    cid      INTEGER,
PRIMARY KEY  (ssn),
FOREIGN KEY (cid) REFERENCES PR-Cands (cid)
)
```

Q1: Can cid be null?

Q2: What if many citizens don't vote?

Q3: Pros and Cons?

Key Constraints



- What about **one** table?
- No! This is bad design.

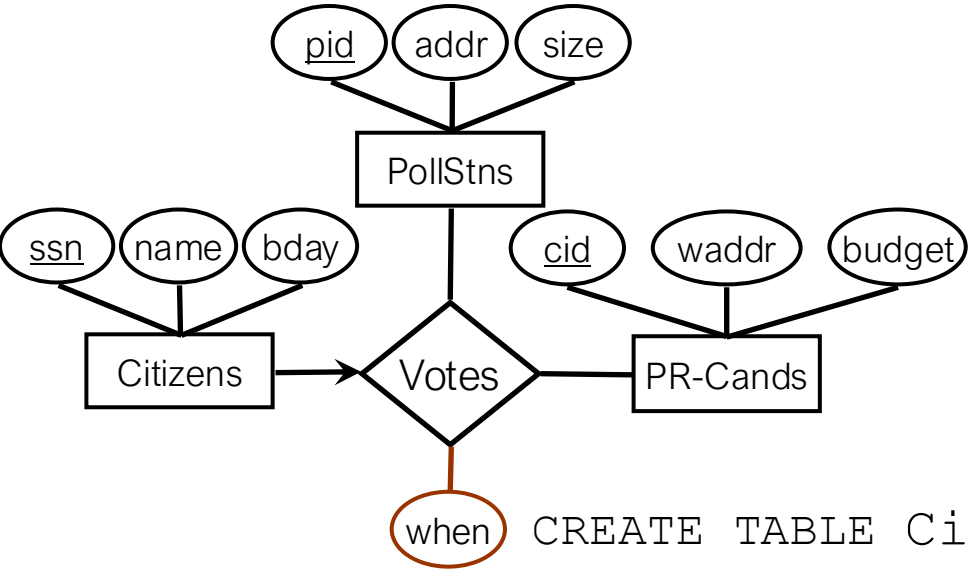
e.g., For each citizen that votes for Ronald Regan, we have to store Regan's information

(cid, waddr, budget) => REDUNDANCY!

Recap: Foreign & Primary Keys

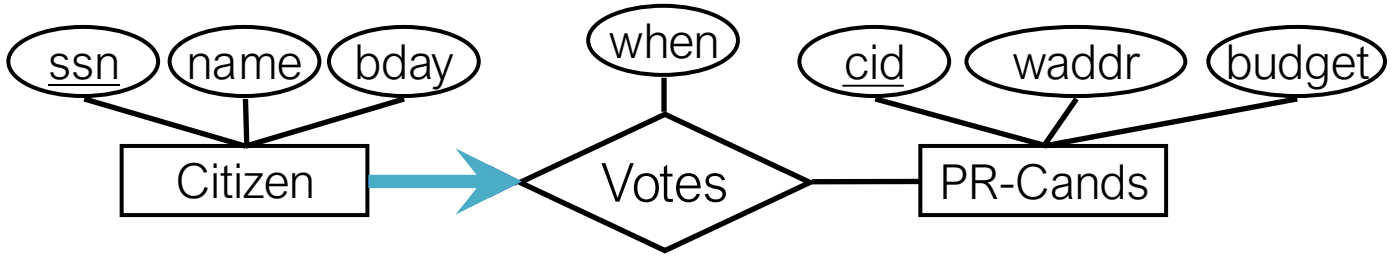
- Foreign key must refer to a single tuple in parent relation
  - Can be a PRIMARY KEY or just a UNIQUE in the parent
- Foreign keys can be NULL in child relation unless part of the child's primary key
- In (most) DBMSs no part of a primary key can be NULL
  - In this class, we will stick with this rule (despite the fact that there are exceptions such as sqlite)
- Although you can choose how to handle UPDATE/DELETE actions for foreign keys, some options may conflict with the primary key (PK) constraints

Key Constraints: N-ary relationships




```
CREATE TABLE Citizens_Votes
(   ssn     CHAR(11),
    name    CHAR(20),
    bday    DATE,
    when    DATE,
    pid     INTEGER,
    cid     INTEGER,
    PRIMARY KEY  (ssn),
    FOREIGN KEY (cid) REFERENCES PR-Cands(cid),
    FOREIGN KEY (pid) REFERENCES PollStns(pid))
```

Participation Constraints

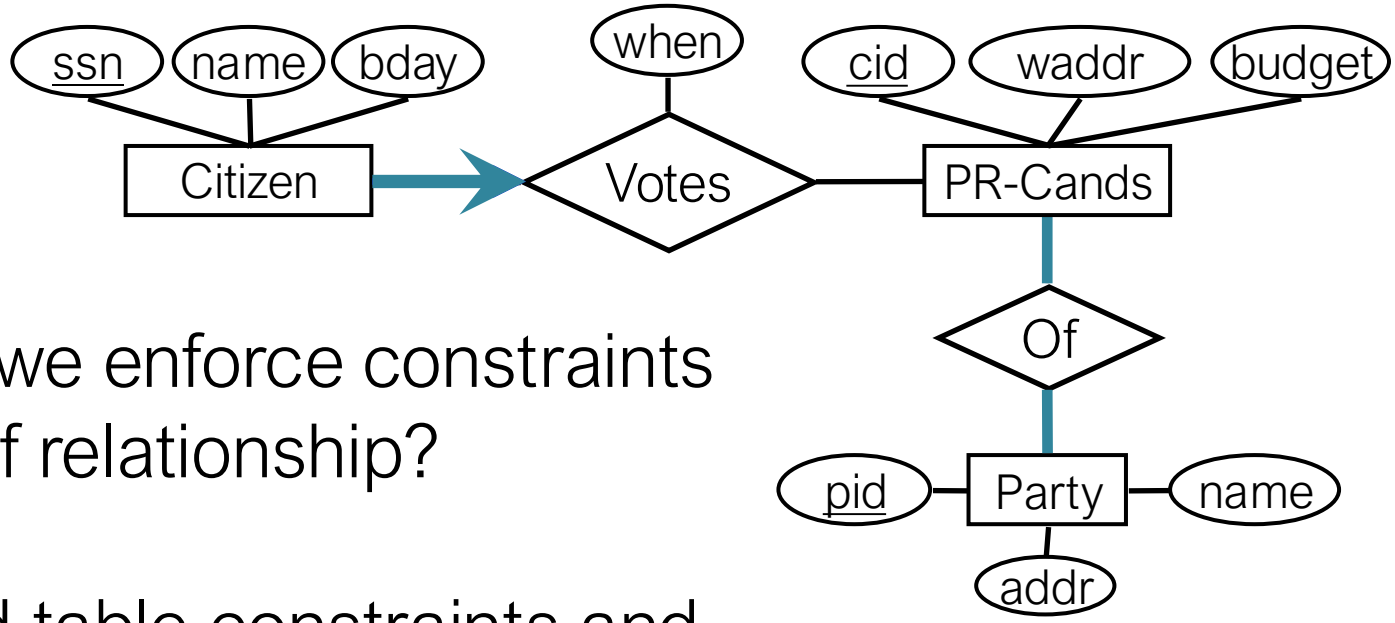


Using Approach 2

```
CREATE TABLE  Citizen_Votes(
    ssn      CHAR(11),
    name     CHAR(20),
    bday     DATE,
    when     DATE,
    cid      INTEGER NOT NULL,
PRIMARY KEY  (ssn),
FOREIGN KEY  (cid) REFERENCES PR_Cands(cid)
   ON DELETE NO ACTION);
```

Can we enforce the participation constraint using Approach 1 (three tables) ?

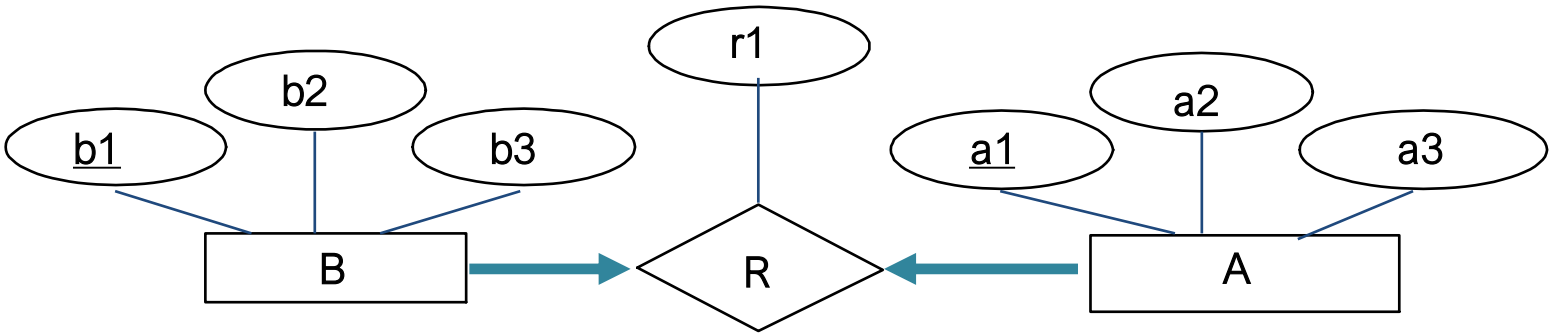
Participation Constraints



Can we enforce constraints on Of relationship?

Need table constraints and assertions (later).

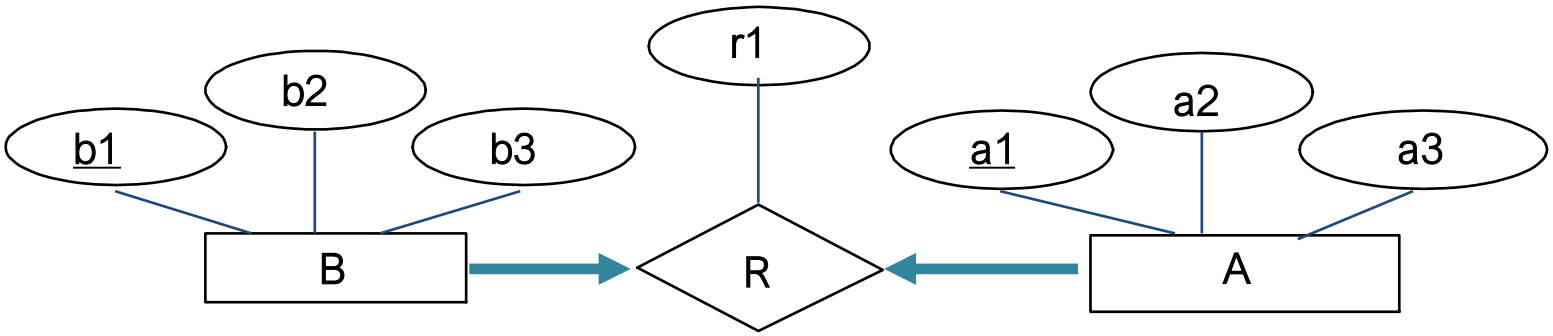
Mapping Participation Constraints (1-to-1)




```
CREATE TABLE  RAB(
        r1 Integer,
        a1 Integer,
        a2 Integer,
        a3 Integer,
        b1 Integer,
        b2 Integer,
        b3 Integer …)
```

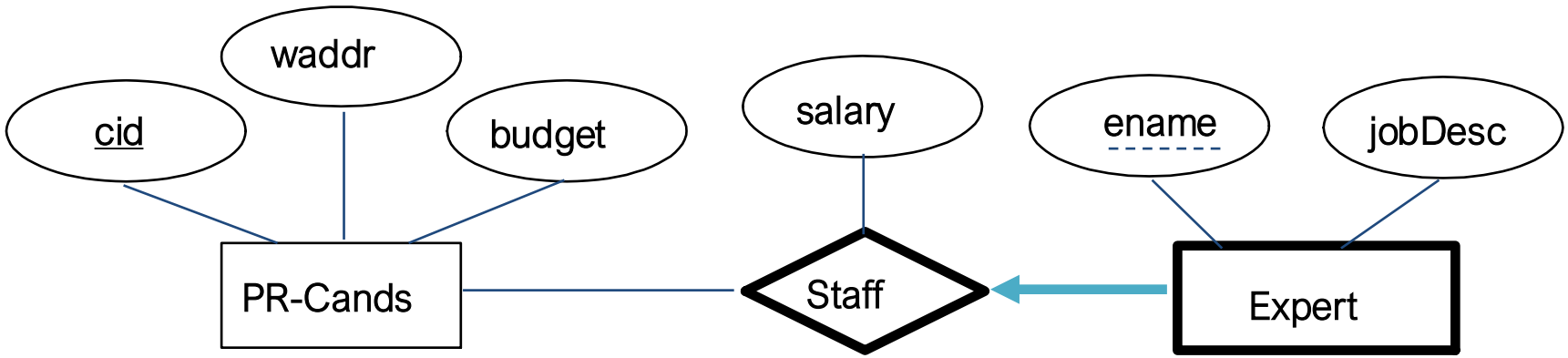
Key constraints?

Mapping Participation Constraints (1-to-1)




```
CREATE TABLE  RAB(
        r1 Integer,
        a1 Integer,
        a2 Integer,
        a3 Integer,
        b1 Integer NOT NULL,
        b2 Integer,
        b3 Integer,
        UNIQUE (b1), PRIMARY KEY (a1))
```

Weak Entities

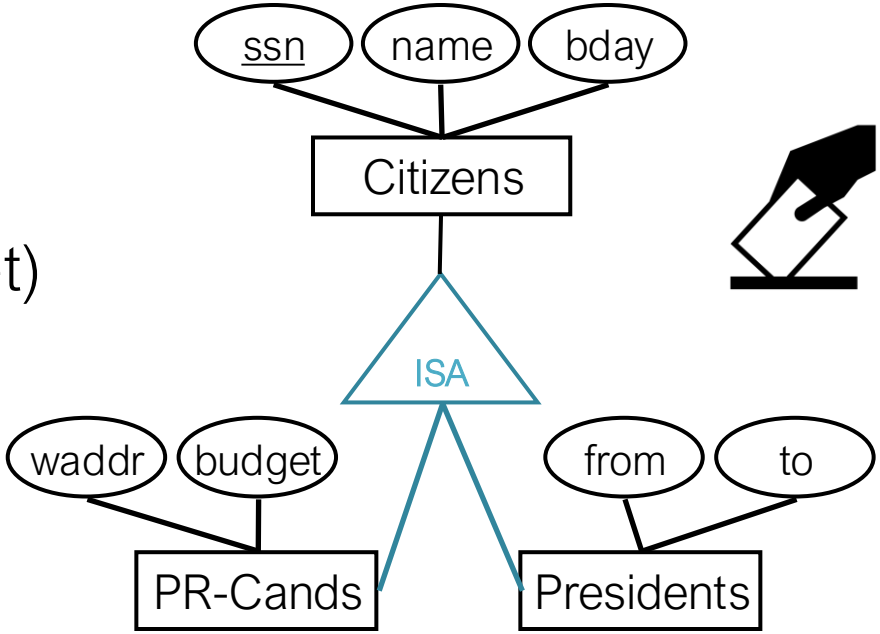


- Approach 2: Combine weak entity and owning relationship into one relation
  - Delete all weak entities when an owner entity is deleted.

```
CREATE TABLE  Expert_Staff (
    ename       CHAR(20),
    jobDesc     CHAR(40),
    salary      REAL,
    cid         INTEGER,
    PRIMARY KEY  (ename, cid),
    FOREIGN KEY  (cid) REFERENCES PR-Cands (cid)
        ON DELETE CASCADE)
```

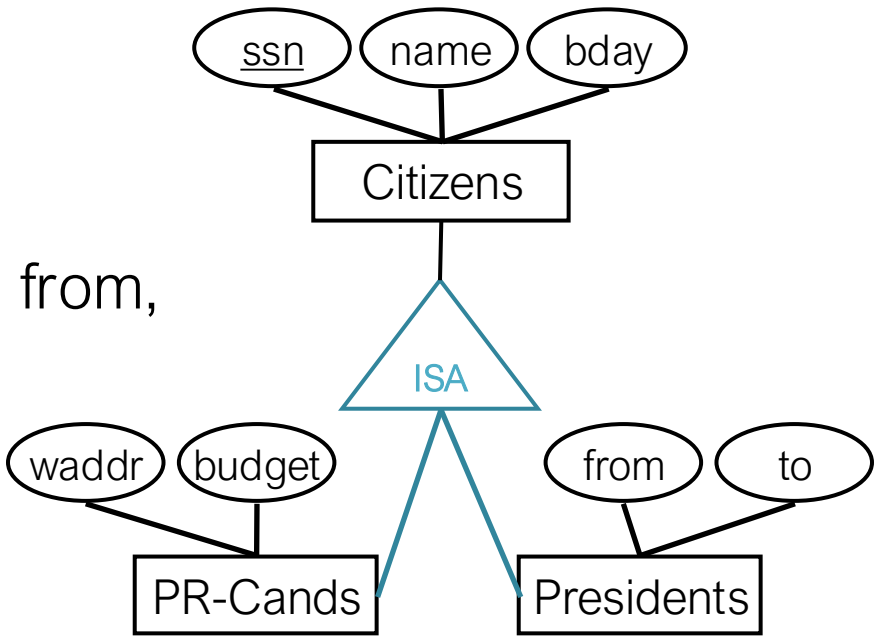
ISA Hierarchies: General Approach

- Three relations:
  - Citizens (ssn, name, bday)
  - PR-Cands (ssn, waddr, budget)
  - Presidents (ssn, from, to)
- Queries:
  - Involving all citizens => Easy
  - Involving just PR-Cands => need to join PR-cands with Citizens to get some attributes



ISA Hierarchies: Alternative

- Two relations:
  - PR-Cands (ssn, name, bday, waddr, budget)
  - Presidents (ssn, name, bday, from, to)

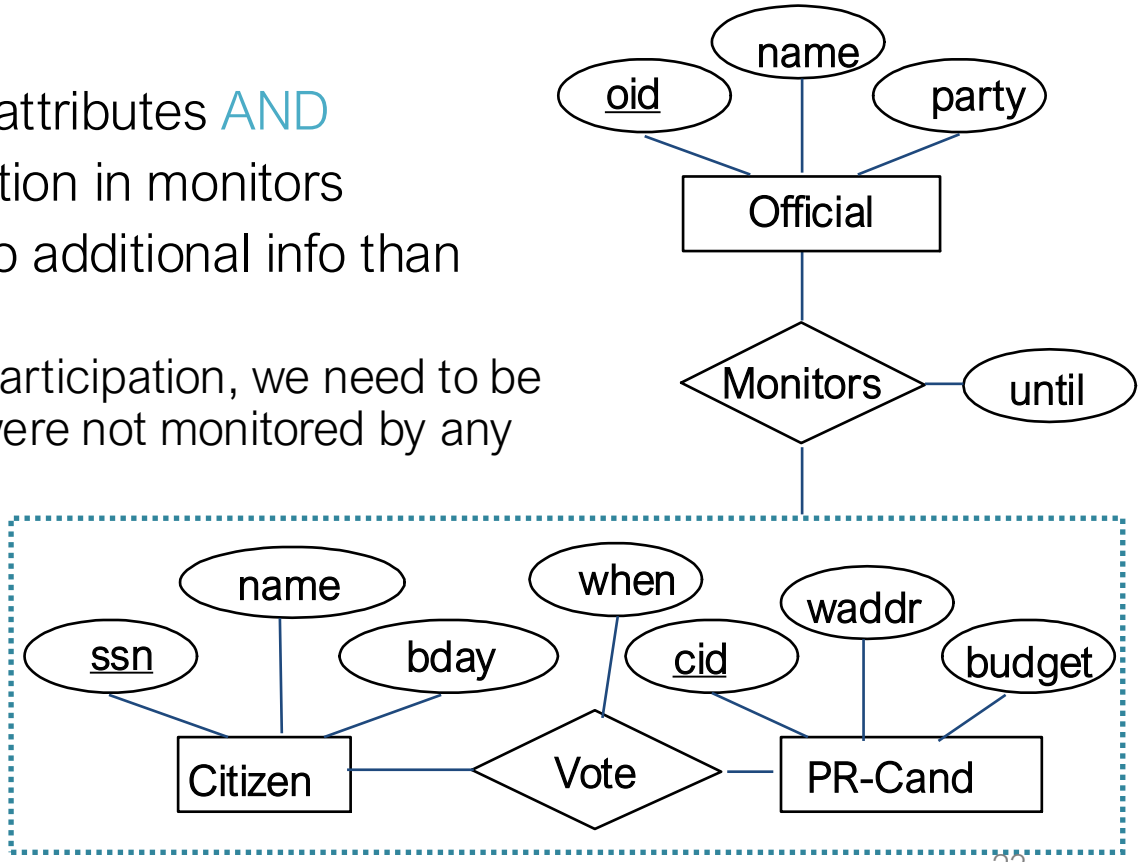


- Problems:
  - What if citizen is both?
    - Redundancy; use general approach if overlapping
  - What if citizen is neither?
    - Use General approach=>This alternative good for covering" ISA

Note: In general, covering & non-overlapping constraints can be expressed only using assertions

Aggregation

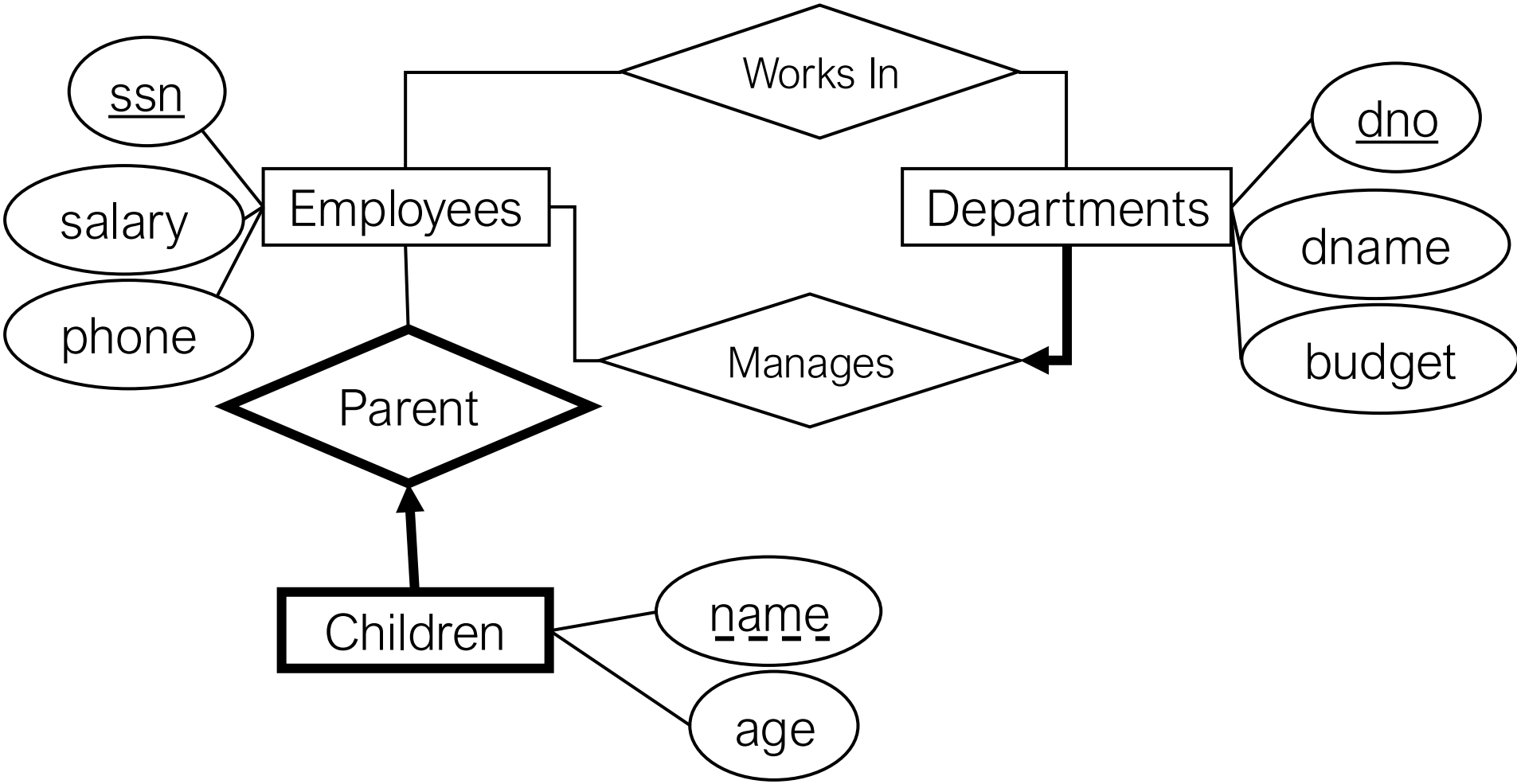
- Keep primary keys of **all** participating entity sets;
  - oid, ssn, cid
- Decide primary
- You can drop Vote only if
  - Vote has no descriptive attributes AND
  - Votes has total participation in monitors
- Why? Because then Votes no additional info than Monitors.
  - If Votes doesn't have total participation, we need to be able to capture votes that were not monitored by any official



Exercise – Part 1

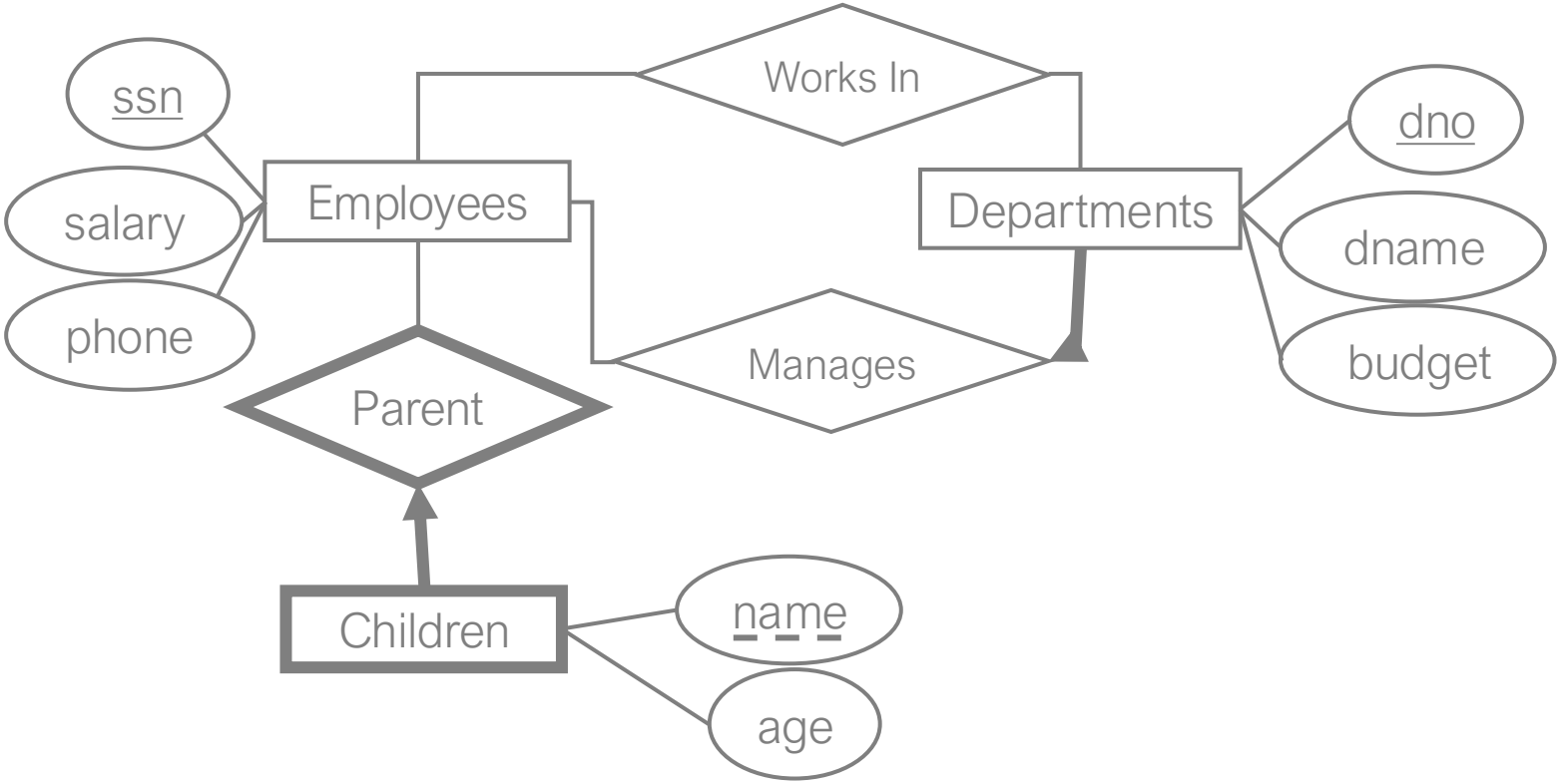
- A **company database** needs to store information about
  - **employees** (identified by <u>ssn</u>, with salary and phone attributes),
  - **departments** (identified by <u>dno</u>, with dname and budget attributes), and
  - **children of employees** (with name and age attributes).
- Employees work in (zero or more) departments
- Each department is managed by exactly one employee
- A child must be identified uniquely by name when the parent (who is an employee; assume only one parent works for the company) is known.
- We are not interested in information about a child once the parent leaves the company.
- Draw an ER diagram that captures this information

ER Diagram (one solution)



Exercise – Part 2

- Write SQL statements to create the corresponding relations, and to capture as many of the constraints as possible.



SQL DDL – One solution

```
CREATE TABLE Employees (
ssn INTEGER,
salary REAL,
phone CHAR(10),
PRIMARY KEY(ssn))
```

```
CREATE TABLE Works (
ssn INTEGER,
dno INTEGER,
PRIMARY KEY (ssn, dno),
FOREIGN KEY (ssn)
   REFERENCES employees (ssn),
FOREIGN KEY (dno)
   REFERENCES departments(dno))
```

```
CREATE TABLE Departments (
dno INTEGER,
dname CHAR(20),
budget real,
manager INTEGER NOT NULL,
PRIMARY KEY (dno),
FOREIGN KEY (manager)
   REFERENCES employees(ssn))
```

```
CREATE TABLE Children (
name CHAR(20),
age REAL,
parent INTEGER NOT NULL,
PRIMARY KEY(name, parent),
FOREIGN KEY(parent)
   REFERENCES employees (ssn)
   ON DELETE CASCADE)
```

Integrity Constraints

- Describe conditions that must be satisfied by every legal instance
- Types of integrity constraints
  - Domain constraints
  - Primary key constraints
  - Foreign key constraints
  - General constraints

Table Constraints (5.7 in book)

- More general than key constraints
- CHECK constraint checked **each time table modified**
- CHECK constraint always true for empty relation

```
CREATE TABLE   Athlete
     (  aid        INTEGER,
        name       CHAR(30),
        country    CHAR(20),
        sport       CHAR (20),
        age         INTEGER,
        PRIMARY KEY   (aid),
        CONSTRAINT  AmericanRule
        CHECK (age >= 17
              OR country <> 'USA')
      );
```

Can also use != but
<> is ISO standard

Try it out in sqlplus or sqlite

```
CREATE TABLE   Athlete
    (  aid       INTEGER,
       name      CHAR(30),
       country   CHAR(20),
       sport      CHAR (20),
       age        INTEGER,
       PRIMARY KEY  (aid),
       CONSTRAINT  AmericanRule
       CHECK ( age >= 17
             OR country <> 'USA')
     );
```

Optional

Most DBMSs only allow simple checks (i.e., no subqueries)

```
INSERT INTO Athletes VALUES (1, 'Mike', 'USA', 'Swim', 16);
```

```
> Error: CHECK constraint failed: AmericanRule
```

Constraints Over Multiple Relations

- For general constraint over multiple tables, use an assertion
- Number of athletes competing in Athens Olympics should be less than 200

```
CREATE ASSERTION letsKeepItPrestigiousRule
CHECK
((SELECT COUNT(*)
  FROM Athlete A, Compete C, Olympics O
  WHERE A.aid=C.aid AND C.oid = O.oid
        AND O.city = 'Athens'
  ) < 200)
```

Practical Considerations

- CHECK with subqueries and ASSERTIONS
  - Part of SQL standard (since 1992?)
  - But, they are not supported by most databases
  - **Main concern**: Performance issues; CHECK or ASSERTION constraints over multiple tables very slow
- Instead: Triggers
  - Most major database systems support them
  - Triggers are procedural (not declarative)

Active Databases & Triggers (5.8 in the book)

Trigger: Procedure that starts automatically if specified changes occur to the DBMS

- Three parts:
  - Event (activates the trigger)
  - Condition (test that is run when the trigger is activated)
  - Action (what happens if the trigger runs)
    - Before and After Triggers
- Trigger Execution
  - Row-level Triggers: Once per modified row
  - Statement-level Triggers: Once per SQL statement

Oracle Trigger Example

```
ALTER TABLE Athlete ADD updated_by char(20);

CREATE TRIGGER trackUSA_AthleteChanges
BEFORE UPDATE ON Athlete        /* Event */
FOR EACH ROW
DECLARE
   v_username varchar2(20);
BEGIN
   IF (:new.country = 'USA') /* Condition */
   THEN                       /* Action */
        SELECT user INTO v_username FROM dual;
        :new.updated_by := v_username;
   ELSE
        :new.updated_by := 'Barzan Mozafari';
   END IF;
END;
/
```

- Example on p170 of textbook has several mistakes (see Canvas for correction)

- This trigger is executed <u>before</u> the activating statement (but you could ask it to run <u>after</u> the activating statement).
- In combination with:
  - `FOR EACH ROW` - execute once per modified record
  - (default) - execute once per activating statement
- Activating statements: `INSERT, DELETE, UPDATE`

Recall CASCADE constraints

```
CREATE TABLE Athlete
(aid INTEGER PRIMARY KEY,
 name CHAR(30),
 country CHAR(20),
 sport CHAR(20));
```

```
CREATE TABLE Olympics
(oid INTEGER PRIMARY KEY,
 year INTEGER,
 city CHAR(20));
```

```
CREATE TABLE Compete
    (aid INTEGER,
     oid INTEGER,
     PRIMARY KEY  (aid, oid),
     FOREIGN KEY (aid) REFERENCES Athlete (aid)
        ON DELETE CASCADE
     FOREIGN KEY (oid) REFERENCES Olympics (oid)
     );
```

CASCADE Using Triggers

```
CREATE TABLE Compete
    (aid INTEGER,
     oid INTEGER,
     PRIMARY KEY  (aid, oid),
     FOREIGN KEY (aid) REFERENCES Athlete (aid)
     FOREIGN KEY (oid) REFERENCES Olympics (oid));

CREATE OR REPLACE TRIGGER cascade_on_delete AFTER
DELETE ON Athlete
FOR EACH ROW
BEGIN
  DELETE FROM Compete
  WHERE Compete.aid = :OLD.aid;
END;
```

Trying out triggers

- Try out the file athlete_trigger_cascade.sql in sqlplus
- Also try it out in sqlite to see if it supports triggers the same way
- Try removing a row from athlete. Does it cascade to Compete via the trigger?
- Try dropping the trigger:
  - `DROP TRIGGER triggername;`
- What happens now on deleting a row from Athlete?

Triggers: Pitfalls and Pain

- Triggers can be recursive!
  - Chain of triggers can be hard to predict, which makes triggers difficult to understand and debug
- Mutating table errors
  - **Mutating table:** A table that is currently being modified by an UPDATE, DELETE, or INSERT statement, or a table updated due to a DELETE CASCADE constraint
  - The session that issued the triggering statement cannot query or modify a mutating table, thus the trigger's code cannot query those tables
  - Used to prevent a trigger from seeing **inconsistent data**

Summary: Triggers vs. Constraints

- Both used to maintain consistency
- Constraints are easier to understand than triggers
  - But many databases do not implement subqueries in CHECK or ASSERTIONs
- Triggers are more powerful
  - Often used to fill out fields in a form
  - Check complex actions (such as credit limit in a shopping application)
  - Check preferred customer status
  - Generate logs for auditing and security checks

Next

Relational Algebra

$$\pi_{sname}((\sigma_{color='red'} Boats) \bowtie Reserves \bowtie Sailors)$$

$$\pi_{sname}(Reserves \bowtie Sailors)$$